

Adv Time Series - Assignment

Pushkar Kale (PRN:1825)

17/05/2020

Objectives

- i. To simulate various non-linear time series models like SETAR, Markov Switching Model, etc.
- ii. To simulate a INAR Poisson Model with the estimation of the parameter by CLS.
- iii. To use Bootstrapping for the estimation of standard error of AR model coefficient.

Question 1

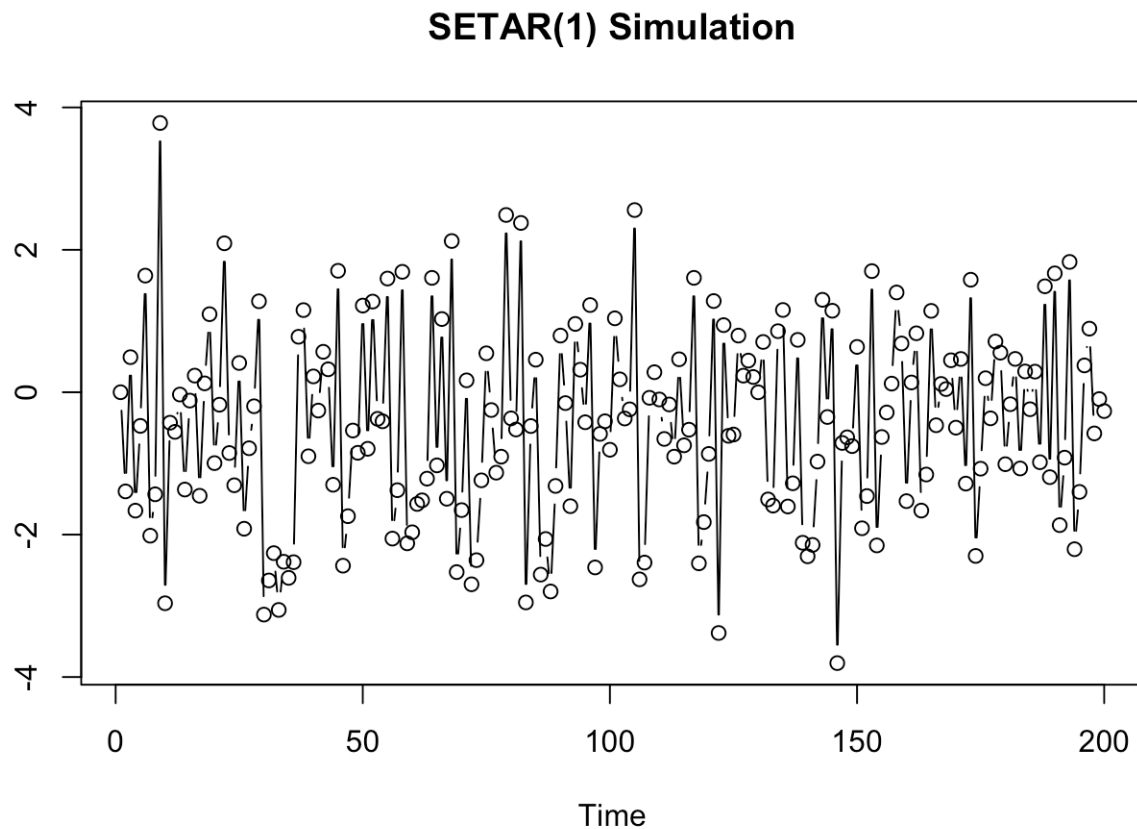
A SETAR model is a piecewise linear model where the parameters of an AR model are determined by the values taken by one or more lagged values of the time series. Consider, for example, the model

$$X_t = \begin{cases} \alpha_1 + \phi^{(1)}X_{t-1} + Z_t & \text{for } X_{t-1} \leq \gamma \\ \alpha_2 + \phi^{(2)}X_{t-1} + Z_t & \text{for } X_{t-1} > \gamma \end{cases}$$

where $\alpha_1, \alpha_2, \phi^{(1)}, \phi^{(2)}$ and γ are constants and Z_t denotes White Noise. This looks like an AR(1) model, but the AR parameter depends on whether X_{t-1} exceeds the threshold ' γ '. This feature makes the model non-linear. As above model depends only on the previous lag, it is called as SETAR(1) model. We will simulate SETAR(1) model with parameters $\alpha_1 = \alpha_2 = 0, \phi^{(1)} = 0.6, \phi^{(2)} = -0.9$ and $\gamma = -1.5$. Hence, our model becomes,

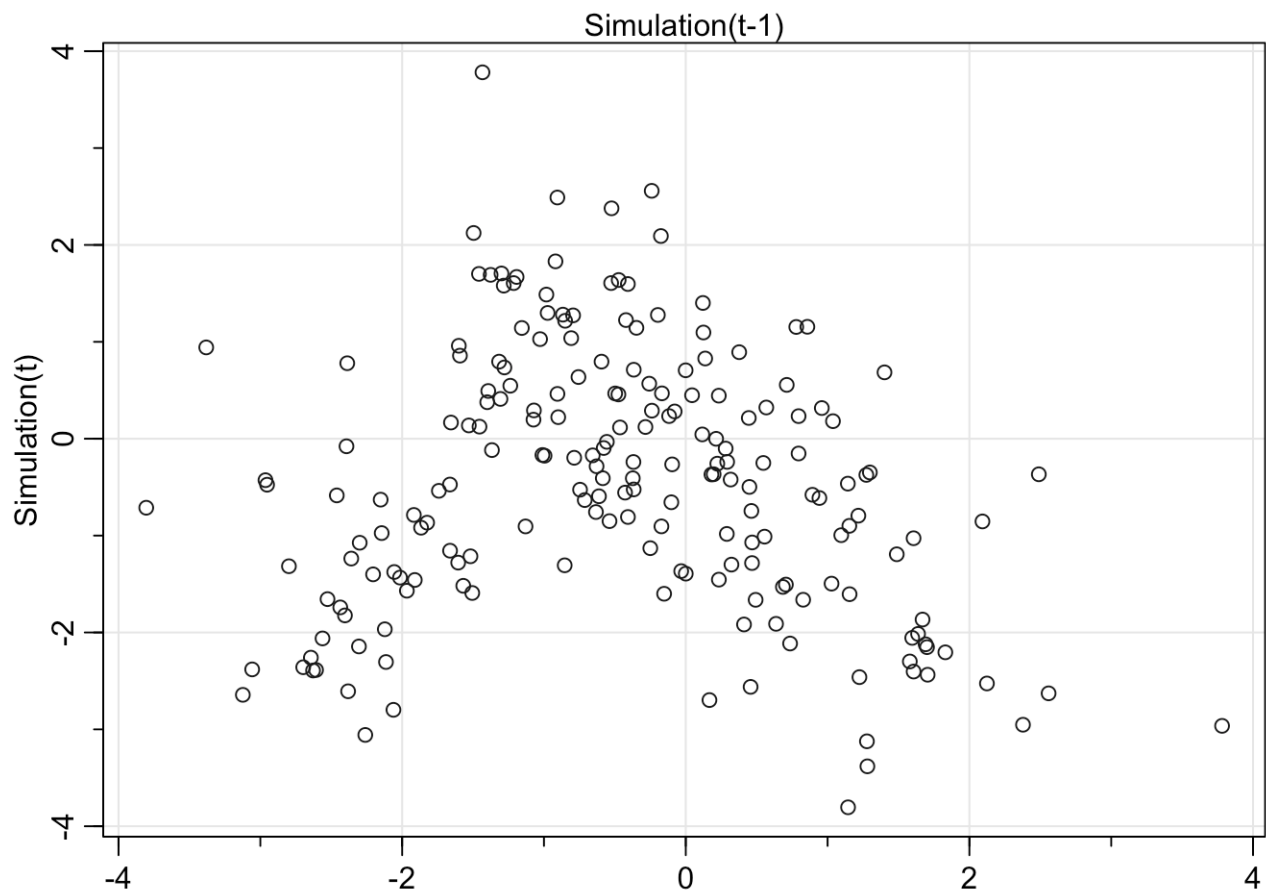
$$X_t = \begin{cases} 0.6X_{t-1} + Z_t & \text{for } X_{t-1} \leq -1.5 \\ -0.9X_{t-1} + Z_t & \text{for } X_{t-1} > -1.5 \end{cases}$$

```
library(tsDyn)    # Required package
Coeff <- c(0,0.6,0,-0.9) # Input of Coefficients
n=200 # Size of Simulation
set.seed(0809) # Setting up the seed
# Actual simulation
Sim <- setar.sim(B=Coeff,lag=1,type="simul",nthresh = 1,Thresh=-1.5,starting=0,
n=n)
Simulation <- Sim$serie # Extracting simulated series
# Plotting the simulated series
plot(Simulation,xlab="Time",ylab="",main="SETAR(1) Simulation",type="b")
```



Now, we need to fit SETAR model for the estimation of parameters of our simulated series. For this purpose, we will use package 'astsa'.

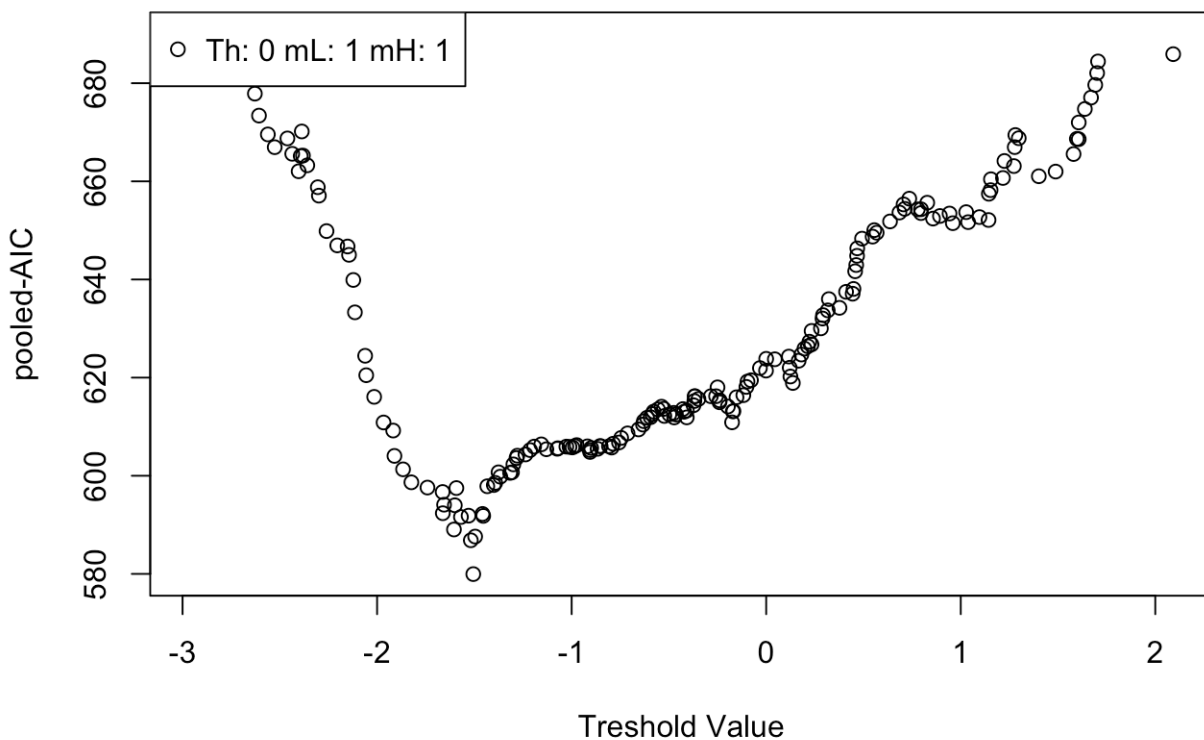
```
library(astsa)
# Plotting series against its first lag
lag1.plot(Simulation,1,corr=FALSE,smooth = FALSE)
```



```
# We get some idea about the threshold from the plot
# Selection of Threshold using AIC
Select <- selectSETAR(Simulation,m=1,th=MakeThSpec(int=c(-3,2)))
```

```
## Using maximum autoregressive order for low regime: mL = 1
## Using maximum autoregressive order for high regime: mH = 1
## Searching within 189 values between -2.963921 and 2.091989
## Searching on 189 combinations of thresholds ( 189 ), thDelay ( 1 ), mL ( 1 ) and MM ( 1 )
```

Results of the grid search



```
print(Select)
```

```
## Results of the grid search for 1 threshold
##      thDelay mL mH      th pooled-AIC
## 1         0  1  1 -1.505667   579.9587
## 2         0  1  1 -1.518285   586.8513
## 3         0  1  1 -1.496009   587.6094
## 4         0  1  1 -1.604944   589.0489
## 5         0  1  1 -1.569071   591.5961
## 6         0  1  1 -1.454588   591.8258
## 7         0  1  1 -1.529791   591.8662
## 8         0  1  1 -1.458307   592.2198
## 9         0  1  1 -1.662259   592.3684
## 10        0  1  1 -1.600420   593.9850
```

From the printed table, we can see that estimated threshold is approximately 1.5, which is also the true threshold value. This is also evident from the plot of AIC vs Threshold value. Now, we would proceed for the fitting of the SETAR(1) model. To fit the model, we need to give AR order for both lower and higher regimes. Here, both the orders are given as 1.

```
Threshold <- Select$bests[2] # Best threshold value
# Here, we need to give AR order for both the regimes
Fit_Model <- setar(Simulation, th=Threshold, mL=1, mH=1)
summary(Fit_Model)
```

```

##
## Non linear autoregressive model
##
## SETAR model ( 2 regimes)
## Coefficients:
## Low regime:
##      const.L      phiL.1
## -0.4421355  0.3514430
##
## High regime:
##      const.H      phiH.1
## -0.1450030 -0.8917958
##
## Threshold:
## -Variable: Z(t) = + (1) X(t)
## -Value: -1.506 (fixed)
## Proportion of points in low regime: 23.62%      High regime: 76.38%
##
## Residuals:
##      Min      1Q   Median      3Q      Max
## -2.63898 -0.72900 -0.12969  0.66087  2.64828
##
## Fit:
## residuals variance = 1.011,  AIC = 10, MAPE = 322%
##
## Coefficient(s):
##
##      Estimate Std. Error t value Pr(>|t|)
## const.L -0.442136    0.652679  -0.6774   0.4989
## phiL.1  0.351443    0.283362   1.2403   0.2164
## const.H -0.145003    0.082725  -1.7528   0.0812 .
## phiH.1  -0.891796    0.081126 -10.9927 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Threshold
## Variable: Z(t) = + (1) X(t)
##
## Value: -1.506 (fixed)

```

From above summary, the estimates of the parameters are as follows:

$\alpha_1 = -0.44$ $\alpha_2 = -0.146$, $\phi^{(1)} = -0.44$, $\phi^{(2)} = -0.89$ Except $\phi^{(2)}$, all other estimates are not close. Increasing the sample size may give more accurate estimates.

If we increase our sample size to $n = 20000$, we get following estimates:

$\alpha_1 = -0.02$ $\alpha_2 = -0.01$, $\phi^{(1)} = 0.61$, $\phi^{(2)} = -0.88$

These are pretty good estimates.

Question 2

Let $\{S_t\}$ be a first order stationary Markov Chain defined on state space $\{1, 2, \dots, k\}$ with $\{\pi_j\}$ as its stationary distribution and (p_{ij}) as its one-step transition probability matrix.

Then, Markov Switching or Regime Switching Model is given by,

$$X_t = \alpha(S_t) + \sum_{j=1}^p \phi(S_t, j)X_{t-j} + \epsilon_t$$

Now, let our state space be $\{1,2,3\}$ i.e. number of states $k = 3$. Also, let $p = 1$. Then our model would be

$$X_t = \begin{cases} \alpha_1 + \phi(1, 1)X_{t-1} + \epsilon_t & \text{for } S_t = 1 \\ \alpha_2 + \phi(2, 1)X_{t-1} + \epsilon_t & \text{for } S_t = 2 \\ \alpha_3 + \phi(3, 1)X_{t-1} + \epsilon_t & \text{for } S_t = 3 \end{cases}$$

We will simulate the model with following parameters:

$$\alpha_1 = 0.7, \alpha_2 = -0.3, \alpha_3 = 0.6, \phi(1, 1) = 0.5, \phi(2, 1) = \phi(3, 1) = 0.4$$

One-step TPM would be

$$\begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.3 & 0.3 & 0.4 \\ 0.4 & 0.5 & 0.1 \end{bmatrix}$$

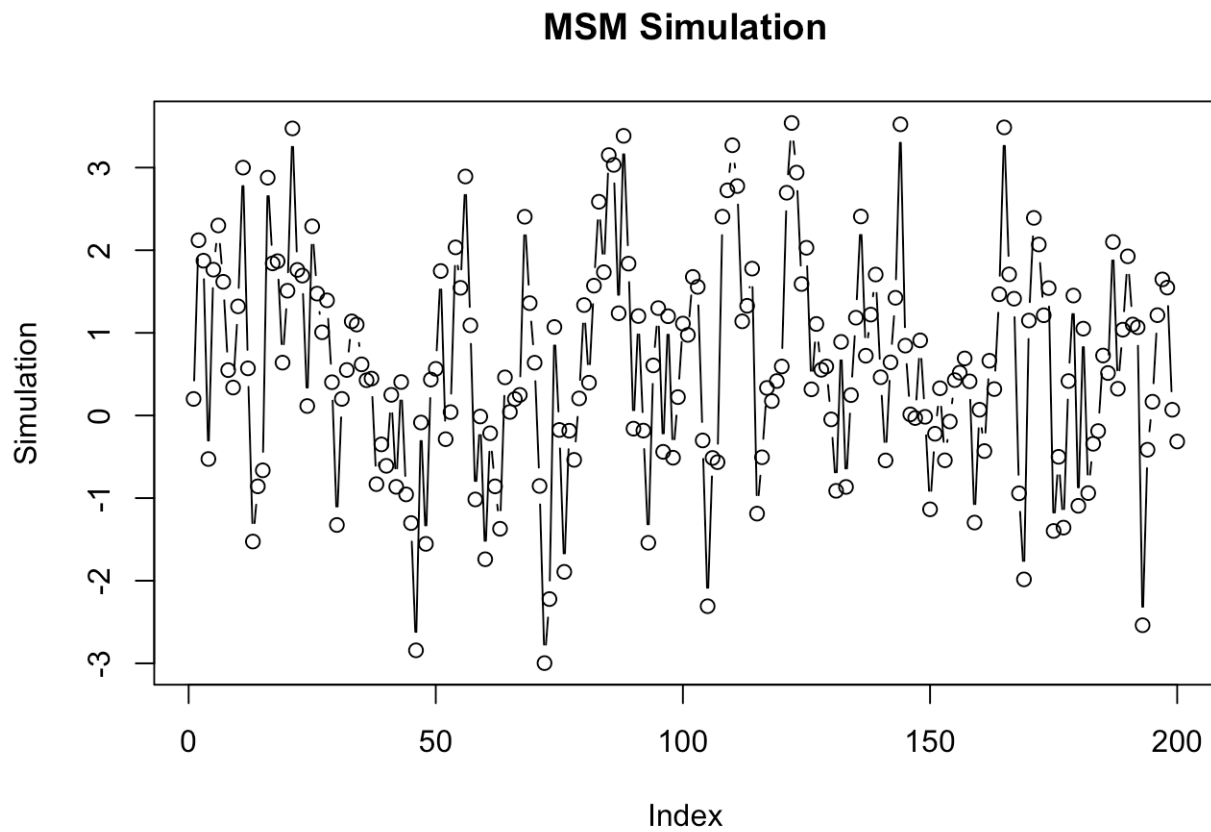
```
# Manual Simulation
S=c(1,2,3) # State Space
# One-step Transition Probability Matrix
TPM=matrix(nrow=3,ncol=3,data = c(0.5,0.3,0.2,0.3,0.3,0.4,0.4,0.5,0.1),byrow=TRUE)
S1 <- c()
S1[1] <- 1 # Initial value of underlying Markov Chain
Simulation=c()
Simulation[1]=0.2 # Initial value of our simulated sample

# parameters
alpha <- c(0.7,-0.3,0.6)
phi <- c(0.5,0.4,0.4)

n <- 200 # Simulation size

for(j in 2:n)
{
  S1[j] <- sample(S,size = 1,prob = TPM[S1[j-1],])
  Simulation[j] <- alpha[S1[j]] + phi[S1[j]]*Simulation[j-1]+rnorm(1) # Actual simulation
}

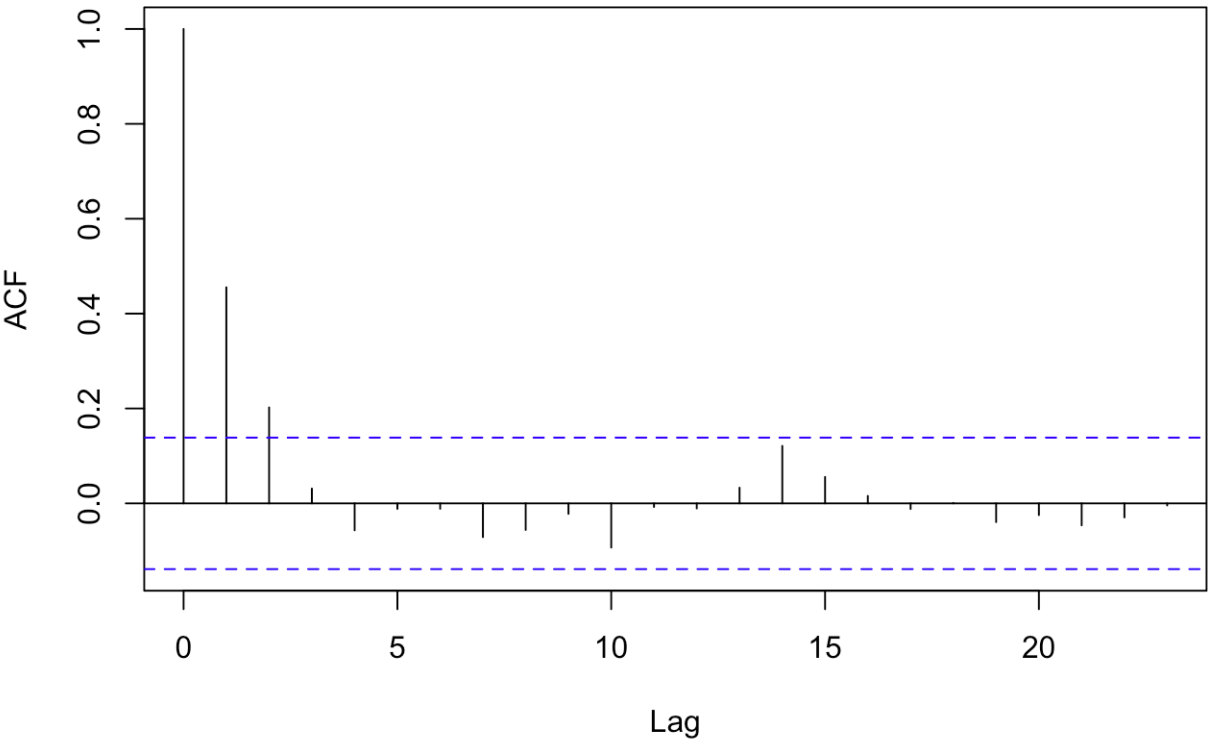
plot(Simulation,type="b",main="MSM Simulation")
```



Now, we would proceed for the fitting of AR(1) model to our simulated series.

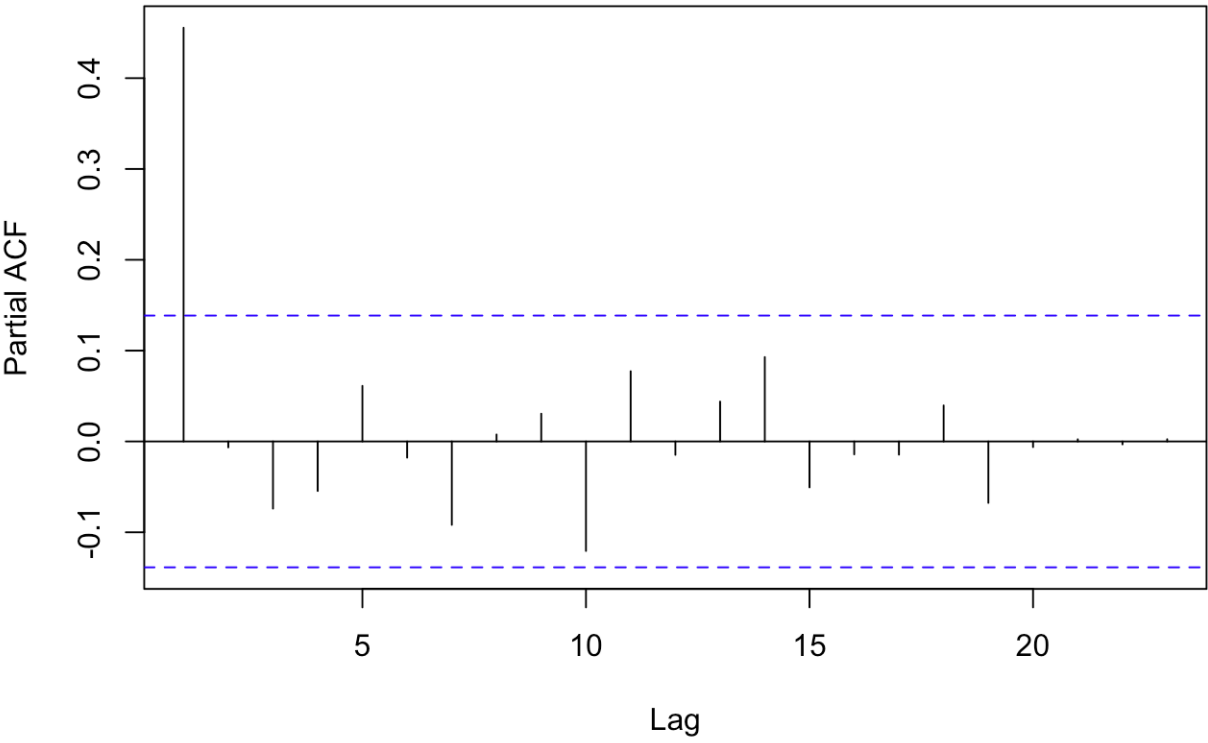
```
# AR fitting  
acf(Simulation, main="ACF of Simulation")
```

ACF of Simulation



```
pacf(Simulation,main="PACF of Simulation")
```

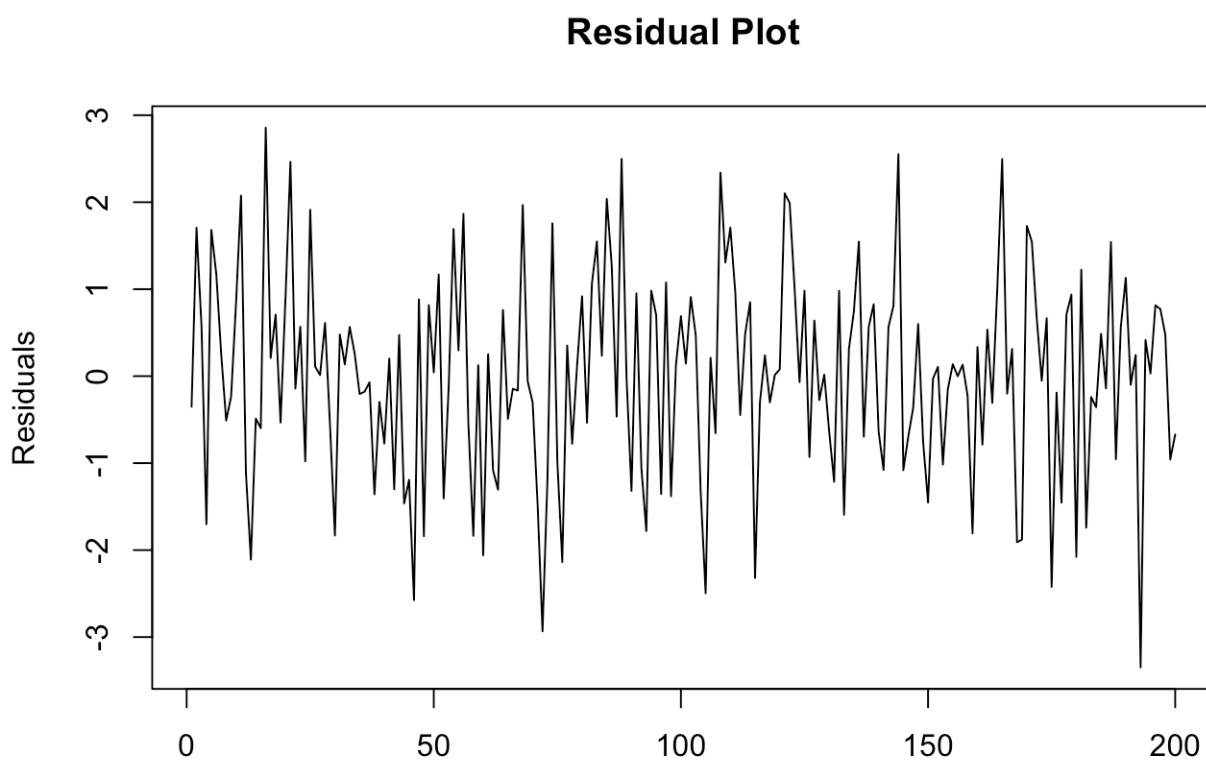
PACF of Simulation




```
# From the acf and pacf, it seems that AR(1),MA(1),MA(2),ARMA(1,2) model may be  
suitable.  
Fit <- arima(Simulation,order=c(1,0,0)) # Fitting of AR(1)  
print(Fit)
```

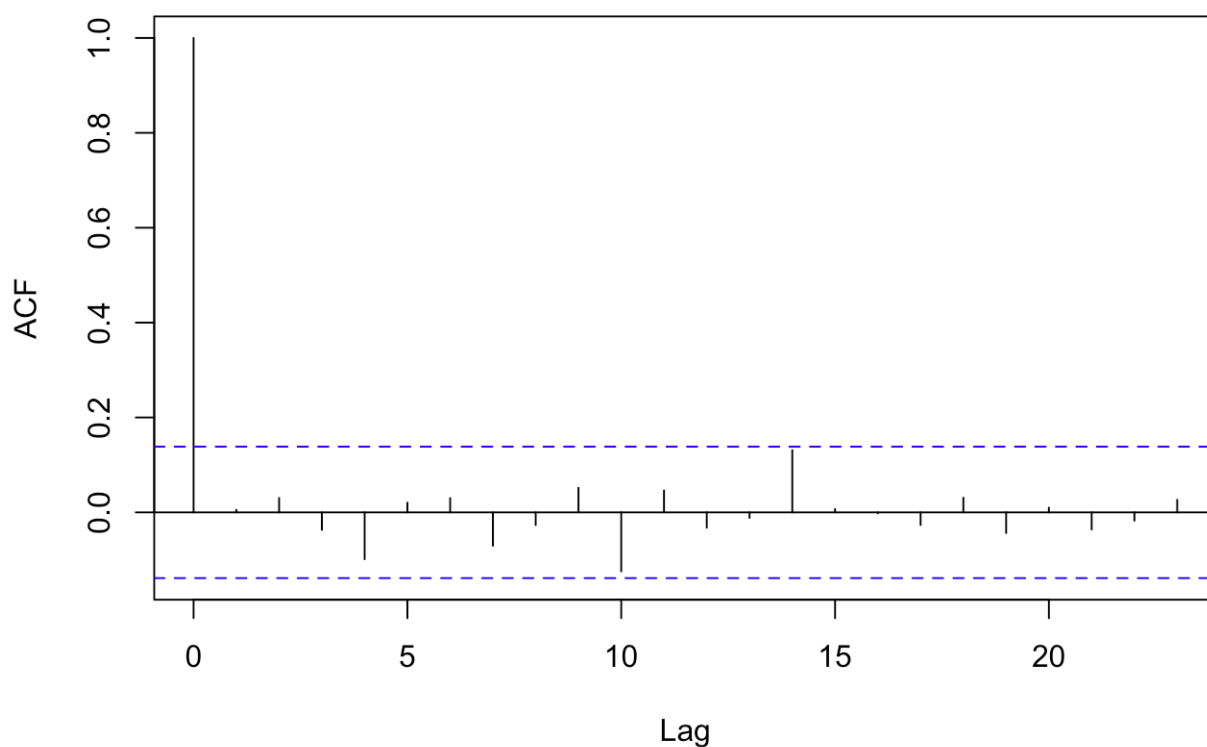
```
##  
## Call:  
## arima(x = Simulation, order = c(1, 0, 0))  
##  
## Coefficients:  
##          ar1  intercept  
##       0.4545    0.5943  
## s.e.  0.0628    0.1506  
##  
## sigma^2 estimated as 1.36:  log likelihood = -314.68,  aic = 635.36
```

```
plot(Fit$residuals,xlab="",ylab = "Residuals",main="Residual Plot")
```



```
acf(Fit$residuals,main="ACF of Residuals")
```

ACF of Residuals



```
Box.test(Fit$residuals) # Box-Pierce Test for residuals
```

```
##
## Box-Pierce test
##
## data: Fit$residuals
## X-squared = 0.0054912, df = 1, p-value = 0.9409
```

AR(1) coefficient is significant. The fitted model is

$$X_t = 0.3843 + 0.4939X_{t-1} + \epsilon_t$$

There is enough evidence of independence of residuals from Box-Pierce Test. Also, acf of residuals seems to be that of white noise. Hence, AR(1) model looks to be a good model.

Question 3

Let X_t be a time series such as $X_t | D_{t-1} \sim \text{Poisson}(\lambda_t)$

where, D_{t-1} : All past observations and $\lambda_t = \alpha + \beta X_{t-1}$

Note that, $E(X_t | D_{t-1}) = \lambda_t$

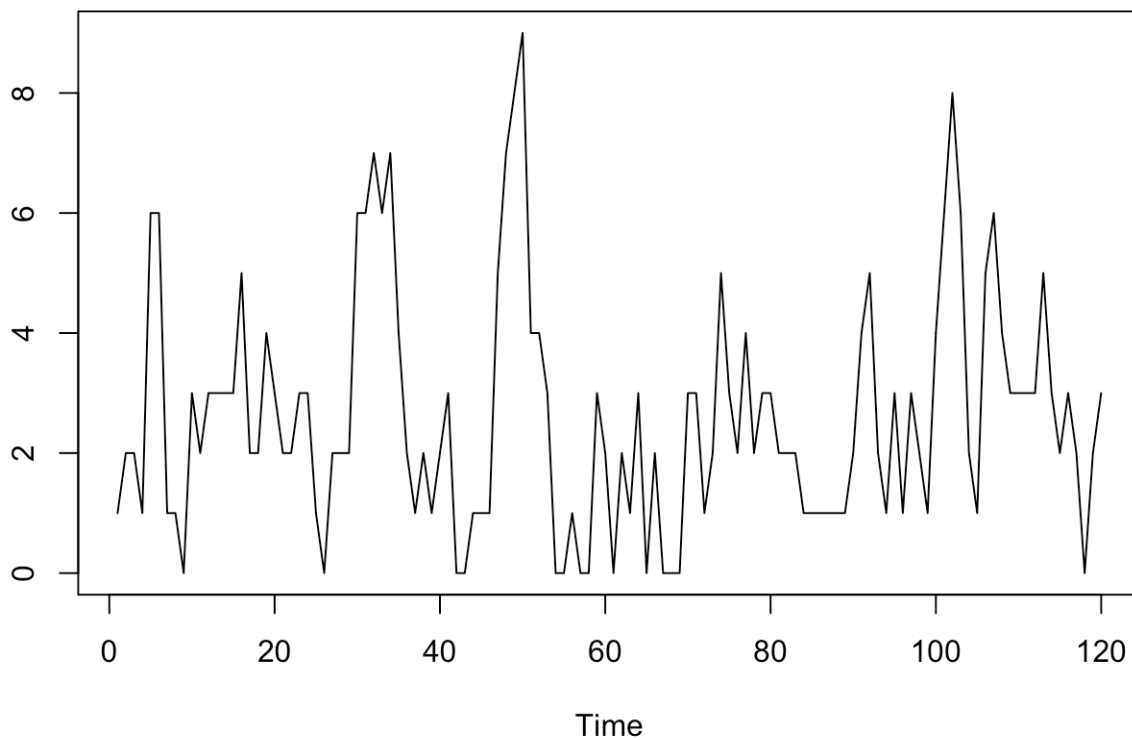
For our simulation, $\alpha = 1, \beta = 0.6$

```

# Simulation
set.seed(777)
n <- 120
Alpha <- 1
Beta <- 0.60
X<-c()
Lam <- c()
X[1]=1
for ( i in 2:n)
{
  Lam[i] <- Alpha + Beta**X[i-1]
  X[i]=rpois(1,Lam[i])
}
plot(X,xlab="Time",ylab="",main="INAR(1) Poisson Simulation",type="l")

```

INAR(1) Poisson Simulation



Now, Conditional Least Square estimator of β is given by the equation,

$$\sum_{t=0}^n (X_{t+1} - \alpha - \beta X_t) X_t = 0$$

This gives,

$$\hat{\beta} = \frac{(\sum_t X_t X_{t+1} - \alpha \sum_t X_t)}{\sum_t X_t^2}$$

```
# By CLS method, estimate of beta is given by sum(X(t+1)*X(t))-alpha*sum(x(t))/
sum(x^2(t))
Prod <- 0
for (i in 1:(n-1))
{
  Prod <- X[i]*X[i+1] + Prod
}
Sum <- sum(X)
Sum_2 <- sum(X^2)

CLS_Beta <- (Prod - Alpha*Sum)/Sum_2
print(CLS_Beta)
```

```
## [1] 0.6030888
```

Our CLS estimate of β is 0.6030888. This is very close to the actual estimate. Now, we will estimate β using Optimal Estimating function.

Let $H_i = X_{t+1} - \alpha - \beta X_t$ be our i^{th} estimating function. Then, optimal estimating function for parameter β is given by,

$$\sum_i H_i W_i^* \text{ where, } W_i^* \propto \frac{E(\frac{\partial H_i}{\partial \beta})}{E(H_i^2)}$$

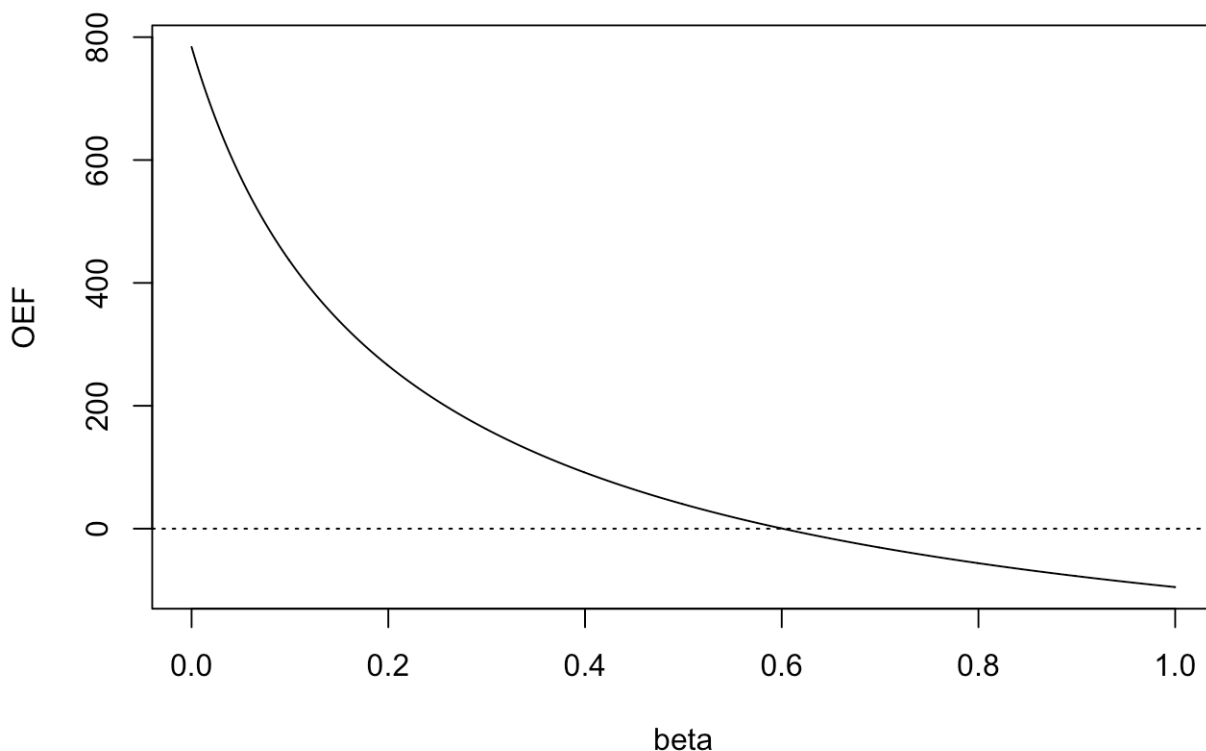
Here, $E(H_i^2) = \text{Var}(H_i) = \alpha + \beta X_t$ and $\frac{\partial H_i}{\partial \beta} = X_t$

Hence, Optimal estimating equation becomes

$$\sum_t (X_{t+1} - \alpha - \beta X_t) \frac{X_t}{\alpha + \beta X_t} = 0$$

```
# Optimal Estimating Function
OEF <- function(beta)
{
  S <- 0
  for (i in 1:(n-1))
  {
    S <- S + ((X[i+1]-Alpha-beta*X[i])*X[i])/(Alpha+beta*X[i])
  }
  return(S)
}
Beta_Vec <- seq(0,1,by=0.001)
OEF_Seq <- OEF(Beta_Vec)
plot(Beta_Vec,OEF_Seq,type="l",xlab="beta",ylab="OEF",main = "OEF Plot")
abline(h=0,lty=3)
```

OEF Plot



```
Beta_Estimator <- Beta_Vec[which(OEF_Seq==min(abs(OEF_Seq)))]
print(Beta_Estimator)
```

```
## [1] 0.601
```

We have plotted OEF as a function of β . We can see that OEF becomes zero at $\beta = 0.601$. Hence, our estimate of β using OEF is $\hat{\beta} = 0.601$. This also seems to be very good estimate.

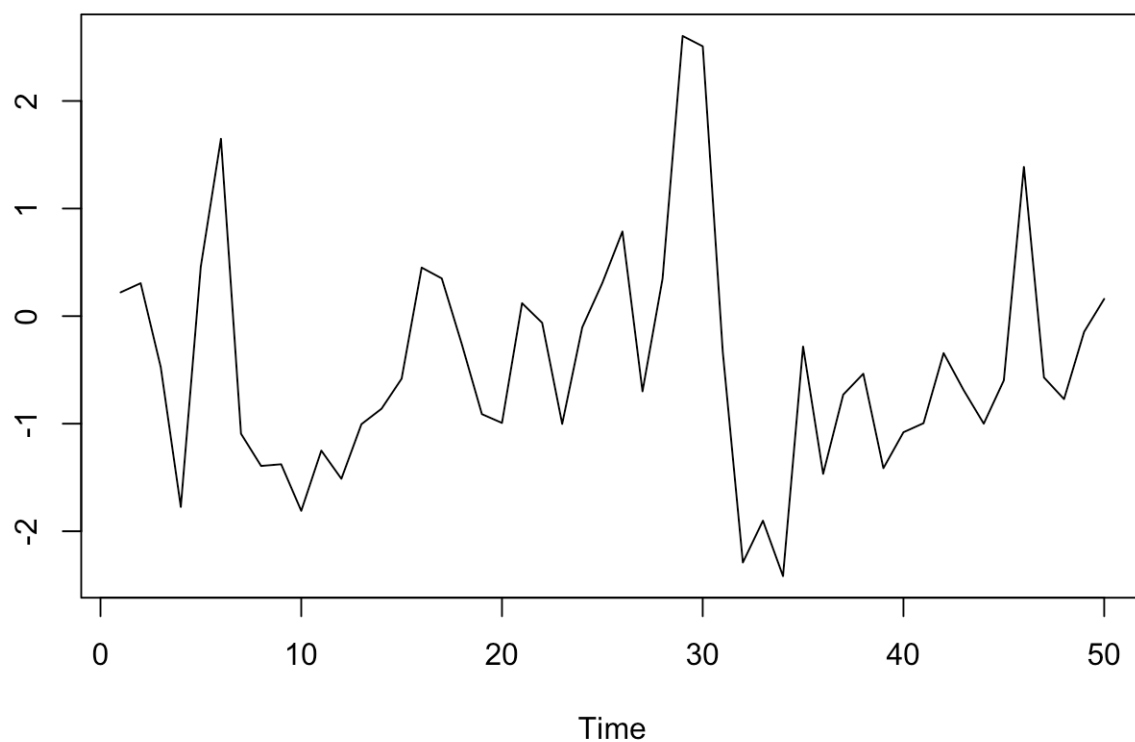
Question 4

Let X_t be a time series such as

$$X_t = \phi X_{t-1} + \epsilon_t$$

Let the residuals be denoted by $r_t = X_t - X_{t-1}$, $t = 2, 3, \dots, T$. Then, the bootstrap procedure is implemented by taking SRSWR of size T from the residuals and then computing random variable X_t^* . This procedure is repeated 'B' times to get the estimate of interested statistic.

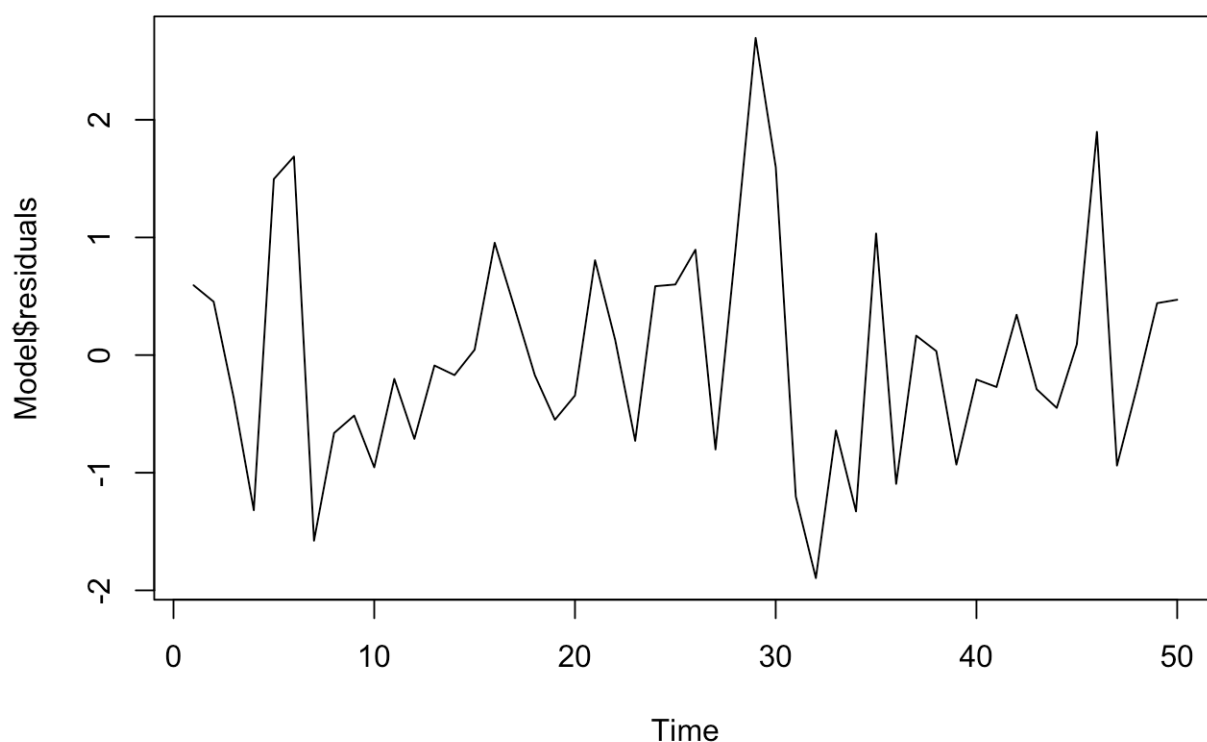
```
## Manual Simulation
n=50
set.seed(77)
Simulation <- arima.sim(model=list(ar=0.77),n=n)
plot(Simulation,type="l",xlab="Time",ylab="")
```



```
Model <- arima(Simulation,order=c(1,0,0))
Model_SE <- sqrt(diag(vcov(Model)))[1] # Standard error of phi
print(Model_SE)
```

```
##          ar1
## 0.1255298
```

```
Coeff <- Model$coef
plot(Model$residuals)
```



```

Residuals <- scale(Model$residuals) # Scaled Residuals
B=100 # No of Bootstrap samples

Obs <- matrix(nrow=n,ncol=B)
Res <- matrix(nrow=n,ncol=B)
Parameter <- c()
for (i in 1:B)
{
  # Intial X*
  Obs[1,i] <- mean(Simulation) + (sd(Simulation)/Model$sigma2)*Residuals[1]
}
for (i in 1:B)
{
  Res[,i] <- sample(Residuals,n,replace = T) # SRSWR from residuals
  for (j in 2:n)
  {
    Obs[j,i] <- Coeff[1]*Obs[j-1,i] + Coeff[2] + Res[j,i] # X*
  }
  M <- arima(Obs[,i],order=c(1,0,0))
  Parameter[i] <- M$coef[1] # AR(1) coefficient
}
Boot_SE <- sd(Parameter) # Standard error
print(Boot_SE)

```

```
## [1] 0.1175281
```

Traditional estimate of standard error of AR(1) coefficient is 0.1256 while, its bootstrap estimate is

0.1175. The difference between the two is not much in magnitude. The same procedure can be implemented directly using 'forecast' package.

References

1. Hyndman, R.J., & Athanasopoulos, G. (2018) Forecasting: principles and practice, 2nd edition, OTexts: Melbourne, Australia. [OTexts.com/fpp2](https://otexts.com/fpp2).
2. STAT 510 - Applied Time Series, Lecture Notes, Pennstate, Eberly College of Science, <https://online.stat.psu.edu/stat510/> (<https://online.stat.psu.edu/stat510/>)
3. Shumway, Robert and Stoffer, David, "Time Series Analysis and Its Applications", Springer, Fourth Edition, 2016.
4. R documentation for package 'tsDyn' - <https://www.rdocumentation.org/packages/tsDyn> (<https://www.rdocumentation.org/packages/tsDyn>)
5. Chatfield, Chris, Xing, Haipeng, "The Analysis of Time Series", CRC Press, Seventh Edition, 2019